Master Git

Animation Link :- https://learngitbranching.js.org/

Animation Link 2 :- https://ohmygit.org/

Notes Link :- https://media.licdn.com/dms/document/media/v2/D561FAQEn6PZtmQsoog/feedshare-document-pdf-analyzed/B56ZxHO9eRKQAY-/0/1770721631742?e=1772064000&v=beta&t=FqG99yTB0cIx6QfsolPa2DYQsBVbaXLpPyVhXBORncc

Git Command Notes Link :- https://media.licdn.com/dms/document/media/v2/D561FAQFTx3etn3ZgLQ/feedshare-document-pdf-analyzed/B56ZxACTI2IsAc-/0/1770600871805?e=1772064000&v=beta&t=JAXVwzeg-XLP2Dg5nbY-LNhtWBFVFWkRv5z686GjzyA

1. Setup & Configuration

Set up your identity before you start working.

Command	Scenario / When to use
`git config --global user.name "Your Name"`	**First-time setup.** Tells Git who you are for commit history.
`git config --global user.email "you@example.com"`	**First-time setup.** Associates your email with your commits.
`git config --list`	**Verification.** Check your current configuration settings.

2. Starting a Project

Initialize a new repo or clone an existing one.

Command	Scenario / When to use
`git init`	**New Project.** You have a local folder and want to turn it into a Git repository.
`git clone <repo_url>`	**Existing Project.** You want to download a copy of a remote repository (e.g., from GitHub) to your machine.

3. The Daily Workflow (Stage & Commit)

The bread and butter of saving your work.

Command	Scenario / When to use
`git status`	**Sanity Check.** Run this *constantly* to see which files are changed, staged, or untracked.
`git add <file>`	**Staging.** You are ready to prepare a specific file for a commit.
`git add .`	**Bulk Staging.** You want to stage *all* changed and new files in the current directory.
`git commit -m "Message"`	**Saving.** You have staged your changes and want to record them permanently in the history.
`git diff`	**Review.** You want to see exactly what lines of code changed *before* you stage them.

4. Branching & Merging

Working on features in isolation.

Command	Scenario / When to use
`git branch`	**Listing.** See all local branches and check which one you are currently on.
`git branch <branch_name>`	**Creation.** Create a new branch (e.g., `feature-login`) without switching to it yet.
`git switch <branch_name>`	**Navigation.** Switch your workspace to a different branch. (Newer alternative to `git checkout`).
`git switch -c <branch_name>`	**Create & Switch.** Create a new branch and switch to it immediately (Shortcut).
`git merge <branch_name>`	**Integration.** You are on `main` and want to bring in changes from `feature-login`.
`git branch -d <branch_name>`	**Cleanup.** Delete a branch after it has been successfully merged.

5. Syncing with Remote

Pushing your code to the cloud (GitHub/GitLab) or pulling updates.

Command	Scenario / When to use
`git push origin <branch_name>`	**Upload.** Send your local commits to the remote server.
`git push -u origin <branch_name>`	**First Push.** Push a new branch for the first time and link it to the remote branch.
`git pull`	**Update.** Fetch changes from the remote server and merge them into your local branch immediately.
`git fetch`	**Safe Update.** Download remote changes *without* merging them (allows you to inspect them first).

6. Undoing Mistakes (The "Oh No" Section)

Fixing errors safely.

Command	Scenario / When to use
`git checkout -- <file>`	**Discard Local Changes.** You modified a file but want to revert it to the last committed state.
`git reset HEAD <file>`	**Unstage.** You accidentally ran `git add` and want to remove the file from the staging area.
`git commit --amend`	**Fix Last Commit.** You forgot to add a file to the last commit or made a typo in the commit message.
`git revert <commit_id>`	**Safe Undo.** You want to undo the effects of a specific old commit by creating a *new* commit that reverses it (safe for shared branches).

7. Inspection & History

Looking back at what happened.

Command	Scenario / When to use
`git log`	**History.** View the list of commits (author, date, message).
`git log --oneline --graph`	**Visual History.** See a cleaner, condensed graph of the commit history and branches.
`git blame <file>`	**Accountability.** See who wrote specific lines of code in a file (useful for debugging).

8. Rebasing (Rewriting History)

Keeps your history clean and linear by moving your commits to the tip of another branch.

> ⚠️ **The Golden Rule:** Never rebase a branch that other people are working on (public/shared branches). Only rebase your local private branches.

Command	Scenario / When to use
`git rebase main`	**Linear Update.** You are on your `feature` branch and want to pull in the latest changes from `main`. Unlike `merge`, this places your work *on top* of `main`, creating a straight line without a "merge bubble."
`git rebase -i HEAD~3`	**Interactive Cleanup (Squashing).** You made 3 messy commits (e.g., "wip", "typo", "fix") and want to combine them into one clean commit before submitting your code for review.
`git rebase --continue`	**Resume.** You hit a conflict during the rebase, fixed the files, and typed `git add`. Run this to proceed to the next commit.
`git rebase --abort`	**Panic Button.** The rebase is getting too messy with too many conflicts. Run this to cancel the operation and return your branch to exactly how it was before you started.

9. Cherry-Picking (Selective Commits)

Copying a specific commit from one branch to another without merging the whole branch.

Command	Scenario / When to use
`git cherry-pick <commit_hash>`	**The "scalpel" approach.** You fixed a critical bug in the `development` branch, and you need to apply *just that specific fix* to the `production` branch immediately, without merging unfinished features.
`git cherry-pick <hash_A> <hash_B>`	**Multi-pick.** You need to copy two or specific non-sequential commits from another branch into your current branch.
`git cherry-pick --continue`	**Resume.** Use this after resolving conflicts caused by the cherry-pick.
`git cherry-pick --abort`	**Cancel.** Use this if the cherry-pick causes issues and you want to back out safely.